Гайд по backend на Python

Что учить и в каком порядке, если нужно дойти до сервиса, который принимает запросы, пишет в базу и запускается не только у тебя на ноутбуке.

```
POST /tasks
Authorization: Bearer ...
Content-Type: application/json

1. проверить токен
2. проверить title и deadline
3. INSERT INTO tasks
4. вернуть 201 и JSON
```

запрос

```
201 Created

{
  "id": 17,
  "title": "..."
}
```

ответ

12 недель Черновик плана для вечеров. Срок можно сдвинуть, порядок — лучше нет.

Один стек Python, PostgreSQL, FastAPI, pytest, Docker. Остальное после первого сервиса.

Три репозитория Скрипт или бот, API задач, тот же API в контейнере с тестами.

Сначала Python и SQL, потом фреймворк

FastAPI на первой неделе прячет функции, ошибки и данные. Их проще разобрать на обычном скрипте.

01
Python
функции, файлы, ошибки

02
SQL
таблицы, JOIN, индекс

03
API
HTTP, JSON, доступ

04
Тесты
pytest и логи

05
Запуск
Docker и сервер

01 ### Скрипт без фреймворка

Читает файл, говорит, какая строка битая, и не падает всем трейсбеком наружу.

02 ### SQL в консоли

SELECT с JOIN пишешь сам. ORM подключаешь, когда запрос уже понятен.

03 ### Один API

Регистрация, список, создание, ошибка в одном формате. Не пять туториалов сразу.

04 ### Репозиторий сразу

Коммит после рабочей штуки. Сообщение «fix» или «update» не считается.

Синтаксис, файлы и ошибки

К концу этапа есть программа, которую можно открыть через неделю и понять, где данные входят и куда уходят.

Разобрать

- Типы, коллекции, условия, циклы
- Функции и области видимости
- Исключения: что ловить, а что пробрасывать
- Классы, когда у данных есть свои правила
- Модули, пакеты, venv
- Типы у функций, которые вызывают другие
- async/await — когда обычные функции уже пишутся без подсказок

Написать

- Список задач в консоли, хранение в JSON
- Чтение CSV: битые строки печатаются отдельно, скрипт не падает
- Бот: /start и одна команда, без базы

Можно идти дальше, если

по трейсбеку находишь строку, а не меняешь код наугад. Чужой скрипт на сто строк можешь пересказать своими словами.

На компьютере

Git, редактор, venv, pip или uv, ruff, pytest. Docker пока не нужен.

Сервер принимает запрос и отдаёт ответ

Между ними проверка данных, правила и запись в базу. Вот это и есть работа backend.

01
Клиент
браузер, бот, curl

02
HTTP
метод, путь, статус

03
Правила
кто что может

04
База
PostgreSQL

Разобрать

- GET, POST, PATCH, DELETE
- Коды 200, 201, 400, 401, 404, 500
- JSON и проверка тела запроса
- Логин и права — разные вещи
- Лог, по которому видно упавший запрос
- FastAPI. Django — если нужна админка сразу

Собрать API задач

```
POST /register
POST /login
GET /tasks
POST /tasks
PATCH /tasks/{id}
```

В README написано, как запустить. Из /docs создаётся задача. Запрос без токена возвращает 401, не стек Python.

База, тесты, сервер

API на SQLite в папке проекта ещё нельзя отдать другому человеку. Нужны нормальная база, проверка сценария и команда запуска.

01

База

- SELECT, JOIN, индекс
- PostgreSQL
- SQLAlchemy после SQL
- Миграции Alembic

02

Проверка

- pytest на сценарий и на ошибку
- Входные данные до записи
- Лог с путём запроса
- Секреты не в Git

03

Запуск

- Процесс и порт
- Dockerfile
- Переменные окружения
- Один прогон в CI

Проект

Запись на слот. Пользователь, время, бронь. Одно время нельзя занять дважды.

В репозитории PostgreSQL, pytest на успешную бронь и на повтор, Dockerfile и README с командами. Без этого проект ещё черновик.

План, если заниматься по вечерам

Срок сдвигается. Порядок лучше не ломать: к концу недели код должен запускаться.

НЕДЕЛИ 1–3

Python

Список задач в файле. Ошибки в данных не роняют скрипт.

НЕДЕЛИ 4–5

Git и SQL

Репозиторий и запросы к PostgreSQL, в том числе JOIN.

НЕДЕЛИ 6–8

API

Регистрация, задачи, /docs, 401 без токена.

НЕДЕЛИ 9–10

Тесты и Docker

pytest на создание задачи. Контейнер поднимается одной командой.

НЕДЕЛИ 11–12

Сервис

Бронь слотов. Другой человек поднимает проект по README.

Три коротких вечера на тему, один длинный — дописать проект. В конце недели коммит и одна строка: что не получилось.

Что смотрят в GitHub

Запуск по README и понятные коммиты важнее списка технологий в шапке профиля.

01

Бот

Команды, пользователи в базе, один внешний API.

02

API задач

Регистрация, роли, PostgreSQL, pytest.

03

Тот же сервис

Docker, лог запроса, полстраницы «как устроено».

В каждом репозитории

- README: зачем проект, как запустить, чем проверяли
- .env.example без настоящих ключей
- Тест на основной сценарий и на ошибку
- Коммиты по делу, не пачка «update»
- Полстраницы, как устроены части
- Ссылка на запущенный сервис или три скриншота

Два репозитория, которые запускаются, лучше одного большого без инструкции.

Если нужна проверка кода

По этому гайду можно идти самому. Если хочешь, чтобы кто-то смотрел решения и говорил, что чинить, напиши. Расскажу, как проходят занятия, и посмотрю, на каком ты сейчас месте.

Что имеет смысл написать сразу

- Писал ли уже на Python и что именно запускал
- Сколько вечеров в неделю реально есть
- Нужен разбор с нуля или уже есть свой проект

Написать в Telegram

Откроется чат @O_V_backend. Текст сообщения уже подставлен: «Привет, интересно узнать про обучение.»